Banking Data Governance & Data Quality Pipeline

Phase 1 and Phase 2 Completion Report

1. Project Overview

The Banking Data Governance & Data Quality Pipeline is a hands-on practice project designed to strengthen practical skills in data governance, data cataloging, data lineage, data quality, SQL, Python, PySpark, OpenMetadata concepts, Agile/Scrum, SDLC, data modeling, and data warehousing.

The project simulates a banking source system and gradually builds toward a governed data engineering pipeline. The current completed work covers repository foundation setup, normalized PostgreSQL source schema design, and Faker-based synthetic banking data generation.

The project is for learning and interview preparation. It is not intended for resume inflation.

2. Working Model

The project is managed using the following working model:

- ChatGPT acts as the project architect, reviewer, and supervisor.
- Codex acts as the implementation engineer.
- Work is not implemented directly without design review.
- Each phase follows this process:
    - Design
    - Review
    - Codex implementation prompt
    - Local validation
    - Commit
    - Phase approval

This approach keeps the project disciplined and closer to real SDLC delivery.

3. Phase 1: Repository Foundation Setup

Phase 1 Goal

The goal of Phase 1 was to create a clean project foundation for the Banking Data Governance & Data Quality Pipeline by copying the old banking pipeline repository and removing or archiving components that were not part of the immediate governance-focused scope.

Phase 1 Work Completed

The old repository was copied into a new repository named:

banking-data-governance-pipeline

The project foundation was cleaned and repositioned from a dbt/Power BI-focused pipeline into a governance and data quality practice project.

Components Archived into legacy/

The following older assets were moved into legacy/ because they are not part of the immediate active implementation scope:

dbt

Kafka

Debezium

Airflow

Snowflake

Power BI-related components

These components were not deleted permanently. They were archived so the active project could remain focused and easier to manage.

Runtime and Generated Folders Removed

Generated/runtime folders were removed from the active working structure to keep the repository clean.

Active Components Kept

The following active components were retained:

data-generator/faker_generator.py

postgres/schema.sql

docker-compose.yml

README.md

docs/project-overview.md

Git Work Completed

Git was initialized and the first project foundation commit was created.

Commit:

c3beab3 Phase 1 - Governance pipeline foundation setup

Tag:

phase-1-complete

Phase 1 Result

Phase 1 successfully created a clean, governance-focused project foundation. The repository was ready for source schema design and synthetic banking data generation.

4. Phase 2: Banking Source Schema and Data Generator

Phase 2 Goal

The goal of Phase 2 was to design and implement a realistic normalized PostgreSQL banking source schema and update the Faker data generator to populate that schema with valid synthetic banking data.

Phase 2 focused only on:

PostgreSQL source schema

Faker-based synthetic data generation

Source data loading into PostgreSQL

The following items were intentionally excluded from Phase 2:

PySpark processing

OpenMetadata installation

Governance YAML artifacts

SQL validation scripts

Curated warehouse modeling

Raw/curated Parquet ingestion

This prevented scope creep and kept Phase 2 focused.

5. Phase 2A: Normalized Source Schema Design

Starting Point

Before Phase 2, the PostgreSQL schema contained only three tables:

customers

accounts

transactions

The existing generator also only created data for:

customers

accounts

transactions

Phase 2A Assessment

The existing 3-table model was too small for a proper banking governance and data quality lab. It lacked realistic source-system relationships, reference data, governance control tables, stable business identifiers, and risk-related data.

Final Phase 2 Source Model

The PostgreSQL source model was extended to 10 tables:

branches

products

reference_codes

customers

accounts

transactions

customer_risk_scores

data_quality_rules

stewardship_issues

source_to_target_mappings

Source Model Purpose

The new schema supports:

Banking source-system simulation

Normalized OLTP-style relationships

Synthetic source data generation

Future data quality validation

Future lineage mapping

Future governance/catalog practice

Future stewardship issue tracking

Key Design Relationships

The normalized relationship model includes:

customers.branch_id -> branches.branch_id

accounts.customer_id -> customers.customer_id

accounts.product_id -> products.product_id

accounts.branch_id -> branches.branch_id

transactions.account_id -> accounts.account_id

transactions.related_account_id -> accounts.account_id

transactions.branch_id -> branches.branch_id

customer_risk_scores.customer_id -> customers.customer_id

stewardship_issues.dq_rule_id -> data_quality_rules.dq_rule_id

Design Decision

The design used both surrogate primary keys and stable business identifiers.

Examples:

branch_id and branch_code

product_id and product_code

customer_id and customer_number

account_id and account_number

transaction_id and transaction_reference

dq_rule_id and rule_code

issue_id and issue_reference

This was done because governance, lineage, cataloging, and source-to-target mapping require stable business-facing identifiers in addition to technical keys.

6. Phase 2B: PostgreSQL Schema Implementation

Phase 2B Goal

The goal of Phase 2B was to update only:

postgres/schema.sql

No generator, PySpark, OpenMetadata, governance files, SQL validation files, or documentation were modified in this step.

Phase 2B Implementation

The PostgreSQL schema was updated to define the full normalized 10-table banking source model.

Implemented tables:

branches

products

reference_codes

customers

accounts

transactions

customer_risk_scores

data_quality_rules

stewardship_issues

source_to_target_mappings

Schema Features Added

The schema now includes:

Rerunnable DROP TABLE statements

Surrogate primary keys

Stable business keys

Foreign key constraints

CHECK constraints

Indexes on foreign keys

Indexes on common lookup/filter columns

Banking-domain columns

Governance support/control tables

Important Schema Capabilities

The schema supports realistic banking concepts such as:

Branches

Products

Customers

Accounts

Transactions

Customer risk scores

Reference codes

Data quality rule definitions

Stewardship issue tracking

Source-to-target mappings

Phase 2B Validation

The schema was loaded successfully into the PostgreSQL Docker container.

Validation command used:

Get-Content .\postgres\schema.sql | docker compose exec -T postgres psql -U banking_user -d banking

Table validation confirmed 10 tables:

accounts

branches

customer_risk_scores

customers

data_quality_rules

products

reference_codes

source_to_target_mappings

stewardship_issues

transactions

Table count validation returned:

10

Phase 2B Commit

Commit:

f2257ca Phase 2B - Implement normalized PostgreSQL source schema

7. Phase 2C: Faker Data Generator Update

Phase 2C Goal

The goal of Phase 2C was to update only:

data-generator/faker_generator.py

The generator needed to populate all 10 normalized PostgreSQL source/control tables with valid synthetic data.

Generator Implementation

The old 3-table looping generator was replaced with a single-run normalized loader.

The generator now:

Connects to PostgreSQL

Clears existing rows safely

Loads tables in parent-to-child order

Uses generated IDs to preserve foreign key relationships

Populates all 10 Phase 2 tables

Prints row counts after each table load

Commits after successful load

Rolls back on failure

Closes database resources properly

Tables Populated by Generator

The generator loads:

reference_codes

branches

products

customers

accounts

transactions

customer_risk_scores

data_quality_rules

stewardship_issues

source_to_target_mappings

Data Generation Logic

The generator creates realistic banking data such as:

Branch records

Banking products

Customer master records

Customer account records

Transaction records

Customer risk scores

Reference code values

Data quality rule definitions

Sample stewardship issues

Sample source-to-target mappings

Foreign Key Validity

The generator preserves valid relationships:

Customers reference real branches

Accounts reference real customers, products, and branches

Transactions reference real accounts

Transfer transactions can reference related accounts

Risk scores reference real customers

Stewardship issues reference real data quality rules

Environment Configuration

The generator requires a .env file in the repository root.

Example values are provided through .env.example.

Required environment variables:

POSTGRES_HOST

POSTGRES_PORT

POSTGRES_DB

POSTGRES_USER

POSTGRES_PASSWORD

The actual .env file is ignored by Git and should not be committed.

Phase 2C Data Load Result

The generator successfully loaded synthetic banking data.

Final row counts:

reference_codes: 72

branches: 10

products: 6

customers: 500

accounts: 990

transactions: 5000

customer_risk_scores: 742

data_quality_rules: 12

stewardship_issues: 8

source_to_target_mappings: 25

Phase 2C Validation

The generator was run successfully using:

python data-generator/faker_generator.py

The final database row count validation showed:

accounts 990

branches 10

customer_risk_scores 742

customers 500

data_quality_rules 12

products 6

reference_codes 72

source_to_target_mappings 25

stewardship_issues 8

transactions 5000

Phase 2C Commit

Commit:

2a03452 Phase 2C - Load normalized synthetic banking source data

8. Phase 2D: Closeout Cleanup

Phase 2D Goal

The goal of Phase 2D was to complete small repository cleanup before tagging Phase 2 complete.

Phase 2D Work Completed

The following files were changed:

docker-compose.yml

README.md

Docker Compose Cleanup

The obsolete top-level version attribute was removed from:

docker-compose.yml

This removed the Docker warning:

the attribute `version` is obsolete

The PostgreSQL service behavior was not changed.

README Update

A short Phase 2 run section was added to the README.

The README now documents the key Phase 2 commands:

docker compose up -d

Get-Content .\postgres\schema.sql | docker compose exec -T postgres psql -U banking_user -d banking

python data-generator/faker_generator.py

The README also notes that .env should be created from .env.example.

Environment File Handling

The project already had:

.env.example

with placeholder PostgreSQL values.

The project also already ignored:

.env

so local secrets/configuration are not committed.

Phase 2D Validation

The following command succeeded:

docker compose config

The PostgreSQL container was confirmed running with:

docker compose ps

Phase 2D Commit

Commit:

ef5944f Phase 2D - Add source data load instructions and compose cleanup

Phase 2 Tag

Tag:

phase-2-complete

9. Final Git History

The current Git history is:

ef5944f (HEAD -> main, tag: phase-2-complete) Phase 2D - Add source data load instructions and compose cleanup

2a03452 Phase 2C - Load normalized synthetic banking source data

f2257ca Phase 2B - Implement normalized PostgreSQL source schema

c3beab3 (tag: phase-1-complete) Phase 1 - Governance pipeline foundation setup

10. Final Repository State After Phase 2

The project now has:

A clean governance-focused repository foundation

A normalized PostgreSQL banking source schema

A working Docker PostgreSQL setup

A working Faker synthetic data generator

Valid generated source data loaded into PostgreSQL

README run commands for Phase 2

Phase 1 and Phase 2 Git tags

11. Phase 1 and Phase 2 Skills Practiced

The completed work practiced the following skills:

Repository cleanup

Project restructuring

Git commits and tags

Docker Compose usage

PostgreSQL schema design

Normalized OLTP modeling

Primary key and foreign key design

CHECK constraints

Indexing

Synthetic data generation with Faker

Python database loading

Environment variable configuration

Data load validation

Source-system simulation

Governance-ready table design

Data quality rule metadata design

Stewardship issue table design

Source-to-target mapping table design

SDLC-style phased delivery

12. What Was Intentionally Not Started Yet

The following items were intentionally not started during Phase 2:

PySpark processing

Raw Parquet extraction

Curated warehouse tables

SQL validation scripts

OpenMetadata setup

Manual governance YAML artifacts

Data catalog ingestion

Lineage visualization

Agile/JIRA simulation artifacts

These are later phases and should not be mixed into the source schema/data generation phase.

13. Phase 2 Final Status

Phase 2 is complete.

Final status:

Phase 1: Complete and tagged

Phase 2A: Schema design complete

Phase 2B: PostgreSQL schema implemented, validated, committed

Phase 2C: Faker generator implemented, data loaded, validated, committed

Phase 2D: Closeout cleanup completed, committed

Phase 2: Tagged as phase-2-complete

The project is now ready for Phase 3.

14. Recommended Next Phase

The next phase should be:

Phase 3: Python Ingestion Layer

The goal of Phase 3 will be to extract source data from PostgreSQL into a raw data zone, likely as local Parquet files first.

Phase 3 should focus only on:

Python extraction scripts

PostgreSQL connectivity

Raw file generation

Load date partitioning

Basic logging

Error handling

Phase 3 should still avoid:

PySpark transformation

OpenMetadata setup

Governance artifacts

SQL validation logic

Curated modeling

Those belong to later phases.